

NDNSF: A DATA-CENTRIC SERVICE FRAMEWORK FOR SECURE, COLLABORATIVE,
AND DYNAMIC NETWORKED APPLICATIONS

by

Tianxing Ma

A Dissertation Proposal

Submitted for Committee Review

Academic Advisor:

Dr. Lan Wang

Major: Computer Science

The University of Memphis

September 2026

Abstract

A requester may know the required service without knowing which authorized Providers are currently available and willing to execute it. The initial NDNSF prototype supports discovery and selection of one Provider per invocation over Named Data Networking (NDN), but not multi-role collaboration. This proposal evaluates that foundation and investigates its extension to dependent service roles.

In the evaluated design, attribute-based encryption (ABE) protects discovery descriptors without enumerating Provider identities. ABE-backed challenge verification, signature validation under trust rules, and request-state checks govern service authorization. Service permissions remain independent of identity-signing credentials; runtime grants and withdrawals require validated permission updates and, for withdrawal, rekeying. Authorized but unselected candidates can read descriptors; request-scoped keys protect selected input and result exchanges.

The proposed collaboration extension uses eligible positive acknowledgment (ACK) Data to assign roles and derive local dependency checks. Providers are intended to accept intermediate Data only from assigned predecessors over declared edges, whether retrieved from producers or caches. The unmanned aerial vehicle (UAV) and distributed-inference applications provide validation settings for invocation, data delivery, runtime authorization, and collaboration.

Existing evaluations support limited invocation and execution-efficiency findings, not the effectiveness of the proposed collaboration mechanism. Planned evaluation compares complete authorization alternatives, tests role and dependency enforcement during execution and recovery, and measures communication, processing, and management costs. The research aims to establish when the invocation design and its collaboration extension are useful, and when their costs outweigh their benefits.

Keywords: Named Data Networking, service authorization, provider discovery, collaboration, distributed inference

Table of Contents

Chapter	Page
1 Introduction	1
1.1 Service Discovery and Authorized Invocation	1
1.2 The Initial NDNSF Foundation	1
1.3 Application Needs and Framework Scope	2
1.4 Extending Invocation to Multi-Provider Collaboration	4
1.5 Thesis and Research Questions	5
1.6 Expected Contributions and Current Foundation	5
2 Related Work and Design Alternatives	7
2.1 NDN Background and Naming Responsibilities	7
2.2 RPC, Robotics Middleware, and Messaging Alternatives	8
2.2.1 Microservices and Edge Computing	9
2.3 Named Computation, Invocation, and Composition	10
2.4 Comparing Authorization Designs under the Same Requirements	10
2.4.1 DNMP as a Concrete Trust-Schema Design	10
2.4.2 Adapting the Alternative to NDNSF	11
2.5 Protected Discovery and Permission Management	12
3 Research Approach: Invocation and Collaboration	13
3.1 Components and Application Responsibilities	13
3.2 Participants, Trust, and Exclusions	15
3.3 Four Named Message Types	16
3.4 Authorization and Confidentiality	17
3.4.1 Rationale for ABE-Backed Service Authorization	17
3.4.2 Authorization Mechanisms, Lifecycle, and Costs	20
3.5 Invocation Modes and Execution Lifecycle	22
3.5.1 Discovery, Known Providers, and Local Composition	22
3.5.2 Admission, Progress, and Failure	23
3.6 Control Messages, Streams, and Complete Objects	24
3.7 Proposed Extension: From ACKs to a Collaboration Plan	25
3.8 Proposed Dependency Admission and Execution State	26
3.9 Conditional Correctness Argument	27
4 Application Scenarios and Validation Plan	28
4.1 Validation Strategy: Applications Exercise the Framework	28
4.2 NDNSF-UAV: Control, Sensing, and Collaboration	28
4.2.1 Application Structure and Service Workflows	28
4.2.2 Operator Workflow and State Interpretation	29
4.2.3 Representative Collaboration: Multi-View Collection and Analysis	31
4.3 NDNSF-DI: Inference Lifecycle, State, and Collaboration	31
4.3.1 Application Structure and Inference Workflows	31

4.3.2	Representative Collaboration: Multi-Provider Inference	32
4.3.3	From a Model Graph to Runtime Assignments	33
4.3.4	Tensor Exchange and Autoregressive State	34
4.4	Coverage, Evidence, and Completion Boundaries	35
5	Preliminary Evidence and Proposed Evaluation	37
5.1	Measured Findings Are Configuration-Specific	37
5.2	Evidence Needed for Each Research Question	39
5.3	Application Experiments and Interpretation	39
6	Research Plan, Feasibility, and Risks	42
6.1	Core Research Scope and Feasibility	42
6.2	Limits That Must Remain Visible	43
7	Expected Outcomes and Proposal Summary	44
	References	44

Chapter 1

Introduction

1.1 Service Discovery and Authorized Invocation

A requester may know the required service without knowing which authorized Providers are currently reachable, capable, and willing to execute it. For example, an operator requesting a timely aerial observation needs a suitable camera Provider, not merely an address at which a camera service was once available. Establishing communication does not settle current willingness, permission, or task completion. The problem is this remaining application decision, rather than host-based communication itself.

Named Data Networking (NDN) supports name-based Data retrieval and data-centric security [1, 2]. Names identify Data, guide Interest forwarding, and participate in trust rules; applications can use a trust schema to constrain acceptable signing keys and guide Data validation [3]. Sync supports discovery of dataset changes so applications can retrieve available Data [4, 5]. NDN-SD builds service-level decisions from these primitives. The detailed networking background is developed in Chapter 2.

Even when one Provider suffices, candidate discovery, authorization, selection, and result retrieval remain application responsibilities. NFN, NFaaS, RICE, and NSC support named computation or invocation [6–9]. The Distributed Network Measurement Protocol (DNMP) provides a concrete example of Trust-Schema-based command authorization [10]. These NDN systems provide the computation, invocation, and authorization precedents against which this proposal examines NDN-SD's design choices.

1.2 The Initial NDN-SD Foundation

The initial NDN-SD prototype selects one Provider from multiple candidates for each invocation. The User publishes Request Data, collects acknowledgment (ACK) Data, and publishes Selection Data for the chosen Provider, which executes and publishes Response Data.

Candidates offer alternatives for the same invocation; the initial prototype does not assign them to different, interdependent roles.

Discovery and authorization remain research subjects independently of collaboration. A discovery descriptor supplies feasibility information, such as deadlines and resource requirements; application input supplies execution material. In the evaluated design, attribute-based encryption (ABE) protects descriptors, while protected-challenge verification, signatures under trust rules, and request-state checks govern authorization. Authorized but unselected candidates can read descriptors. The study also evaluates input/result protection, permission grants and withdrawals, and management costs.

1.3 Application Needs and Framework Scope

A networked application often needs more than the location of a service. A drone operator may require a camera near an area before a deadline, while an inference requester may require a particular model, sufficient accelerator memory, and an acceptable preparation delay. A service advertisement identifies a possible source of execution, but it does not establish that the source is willing and able to undertake this task now. NDNSF separates permission to provide a service, current willingness to participate, selection for an invocation, and completion of the requested work.

These distinctions also matter within an application that usually uses one device. After choosing a drone, an operator expects subsequent camera commands to affect that drone. Substituting another available Provider would change the meaning of the command. Conversely, an image-analysis request may permit any authorized compute Provider that meets its requirements. A reusable framework must support both situations without requiring the application to implement an unrelated communication protocol for each.

The discovery descriptor contains the information needed for a participation decision. For a sensing task, this may include a coarse area, a time window, and a sensor requirement. For inference, it may include model identity, expected input size, and a deadline. Full images,

detailed flight instructions, and inference prompts can be delivered after selection. The boundary follows the decision: information indispensable to assessing feasibility cannot simply be withheld until after that assessment. An application should nevertheless disclose only the necessary precision. For example, an authorized camera Provider may need an approximate search area, while an unselected candidate need not obtain the full mission route. Descriptor protection limits disclosure to authorized service Providers; it does not prevent them from learning or sharing what they legitimately decrypt.

NDNSF-UAV and NDNSF-DI expose complementary demands. The former combines device-specific operations, changing observations, live data, and physical actions. The latter combines compute selection, model preparation, large artifacts, intermediate tensors, and incremental outputs. Both require authorization and bounded request lifetimes. Their differences test whether a common invocation and named-object interface remains useful beyond one workload. Reusability is an engineering objective to be evaluated through shared mechanisms and application-specific code, rather than assumed from implementing two demonstrations.

The scope is organized around an invocation’s lifecycle: deciding who may participate, deciding which work may start, and obtaining an interpretable outcome before the operation closes. These questions connect the application cases to prior invocation mechanisms [8, 9] and command authorization [10]. Table 1 groups the resulting requirements. It is a bounded scope for the workflows studied here, not an exhaustive inventory of every possible service framework. For example, billing and hard real-time flight control require additional application mechanisms. RQ1 examines participation and permission decisions, RQ2 examines dependencies when work spans Providers, and RQ3 examines observed outcomes and costs.

These are related requirements, not four strictly separated protocol layers. Authorization affects every remote operation, and execution state determines whether an otherwise valid object may be used. The following chapters explain the NDN foundation, the framework mechanisms, and their application-level validation. The research questions retain a narrower evidential purpose: determine which choices are justified, which additional guarantees the collaboration

Table 1: Framework requirements and their relationship to the research questions.

Requirement category	Required behavior	Research link
Discovery and invocation	Find willing candidates, select an executor, correlate completion or timeout	RQ1, RQ3
Authorization and confidentiality	Check both service permissions, restrict Content access, apply permission changes	RQ1, RQ3
Execution and collaboration	Associate work with active requests; enforce assigned roles and dependencies in the extension	RQ2, RQ3
Named-object delivery	Distinguish control messages, ongoing streams, and complete artifacts; validate retrieved Data	RQ2, RQ3

extension provides, and what their costs are.

1.4 Extending Invocation to Multi-Provider Collaboration

Some tasks require complementary services from different Providers. Independently invoking each service does not by itself specify which Provider fills each role or which intermediate objects a dependent service may accept. This proposal investigates an extension that makes those assignments and dependencies explicit while retaining the invocation foundation.

For example, nearby unmanned aerial vehicles (UAVs) could collect different views of a ground area for analysis by a compute Provider. The extension would assign sensing and processing roles and check each image against its declared dependency. Area and deadline information may be needed before ACK to assess feasibility. After Selection authorizes the sensing tasks, the UAVs produce images for the assigned compute Provider; Selection supplies execution details and the required access information, not images that have yet to be captured. Descriptor encryption limits reading to authorized candidates, not exposure of names or traffic patterns. This case motivates the extension; recognition algorithms remain an application choice.

1.5 Thesis and Research Questions

The thesis to be evaluated is that named, verifiable objects can support runtime service discovery and authorized single-Provider invocation, and that explicit role and dependency rules can extend this foundation to multi-Provider tasks. The study evaluates the discovery/authorization design and the collaboration extension separately. Benefits must justify their discovery, validation, communication, and key-management costs under the tested conditions.

- RQ1. How can NDN SF discover willing Providers while keeping discovery descriptors confidential from unauthorized readers and checking service permissions before execution, and at what cost relative to complete alternative designs?
- RQ2. How can the single-Provider invocation foundation be extended so Providers selected at runtime follow task roles and named dependencies, whether intermediate Data is retrieved from producers or caches?
- RQ3. Under what workload and network conditions do these mechanisms improve completion, latency, or execution efficiency, and when do their costs outweigh their benefits?

RQ1 evaluates the invocation foundation, including a DNMP-inspired role-authorization alternative paired with content encryption and adapted to the NDN SF workflow. RQ2 investigates the collaboration extension through UAV and distributed-inference dependencies; RQ3 measures both scopes and their applicable failure behavior separately.

1.6 Expected Contributions and Current Foundation

The intended contributions are an evaluated discovery/authorization design, a collaboration protocol enforcing runtime role assignments through named-dependency checks, and evidence of their benefits and costs. These build on established NDN, encryption, and computation mechanisms.

Existing invocation experiments do not validate a multi-role plan. The extension adds role and dependency rules without replacing existing request-ID isolation. Later implementation work is not attributed to the initial prototype; collaboration end-to-end evaluation remains incomplete. Chapter 5 separates available findings from required evidence.

The proposal seeks committee agreement on these research questions, their significance, and a feasible method for answering them. Chapter 2 establishes the background and alternatives. Chapter 3 presents the approach, separating the invocation foundation from the proposed extension. Chapter 4 defines application scenarios and validation coverage. Chapter 5 distinguishes preliminary evidence from proposed experiments, and Chapter 6 specifies the remaining work, risks, and timeline. Completed implementation supports feasibility; it does not replace evaluation of the research claims.

Chapter 2

Related Work and Design Alternatives

2.1 NDN Background and Naming Responsibilities

NDN retrieves Data by name using Interest and Data packets [1, 2]. An application expresses an Interest describing the Data it seeks. A matching Data packet can be supplied by its producer or an intermediate cache. Forwarders use forwarding information to direct Interests and pending-Interest state to deliver returning Data. This permits network-layer reuse of named Data without requiring the consumer to identify its present storage location. Availability still depends on forwarding, connectivity, and a reachable copy; a name is not a delivery guarantee.

Names have several related uses in this proposal. A service name identifies the requested operation. A message name, together with the message’s validated fields, identifies an invocation exchange. An object name identifies data such as an image, model fragment, or output tensor. Names also participate in trust rules that constrain acceptable signing keys [3]. These uses should agree, but should not be conflated: matching a service prefix does not by itself prove service permission, and a valid producer signature does not prove that the object is the input assigned to the current execution.

For example, an illustrative image name could have the form

`/org/uav7/view/task42/view2/version1`

under the sensing UAV’s producer namespace. The application must define what the task, view, and version components mean and validate the signer. A processing Provider can refer to that exact image rather than rename it as its own observation. This example explains the naming principle, not a mandatory NDNSF wire format. Names and traffic patterns can remain visible even when Content is encrypted. Sensitive identifiers should therefore not be embedded in public names merely because the associated payload is protected.

The forwarding Content Store offers opportunistic caching. A repository provides an application-managed place to retain objects for later retrieval. Neither a cache hit nor a successful

digest check supplies the complete admission decision for a dependent service. The consumer still needs the producer’s authority, the intended object identity, and the current invocation or collaboration association. Likewise, a Data packet’s FreshnessPeriod controls cache freshness treatment; it is not a substitute for message expiry, current permission checks, or application replay state.

Sync helps participants learn about changes in a named dataset [4, 5]. Applications can then retrieve and validate the announced Data. Thus, publication in NDNSF means making a named object available and announcing it through the configured dissemination mechanism. It should not be read as an unsolicited push of a Data packet through arbitrary forwarders. Sync recovery can repair missing announcements or trigger retrieval while time remains. It cannot ensure completion after an application deadline or during an indefinitely disconnected interval.

These distinctions define the meaning of data-driven execution here. A validated Request, ACK, Selection, result, or declared intermediate object supplies evidence for a local state transition. Timers and local scheduling policies still exist. A timeout can close collection or terminate an operation, and a planning algorithm can decide which assignment to publish. The protocol-relevant decision is represented in named, verifiable messages, so remote participants do not have to infer it from an unrecorded local callback. This does not imply that all runtime state is network Data or that packet arrival alone authorizes execution.

2.2 RPC, Robotics Middleware, and Messaging Alternatives

RPC frameworks provide typed service interfaces, and gRPC supports both unary and streaming calls [11]. Robotics middleware such as ROS provides reusable application communication abstractions [12, 13]. Message-oriented middleware supports decoupled producers and consumers [14]. These are relevant alternatives to writing application-specific networking. NDNSF does not claim that discovery, asynchronous interaction, or distributed application composition is absent from existing middleware.

The comparison concerns how the same application requirements are expressed. An

RPC experiment must account for service resolution, connection management, retries, and failure detection in the selected implementation. An NDN experiment must account for name discovery, synchronization, retransmission, validation, and Content Store behavior. A typed call is not inherently less secure than named-object exchange; security depends on the accompanying authentication, authorization, and encryption mechanisms. Similarly, network-layer caching is useful only where object reuse and the workload permit it. Reusing an image does not mean that an invocation with physical side effects may be repeated safely.

For NDNF, the design opportunity is to connect service decisions and application data through names and verifiable objects. The evaluation must test the resulting benefits rather than equate a difference in networking abstraction with superiority. Related named-computation and command-authorization systems below provide more direct protocol precedents.

2.2.1 Microservices and Edge Computing

CFEC studies computation offloading across vehicular fog, edge, and cloud resources using mobility and resource information [15]. MIA-NDN studies microservice-aware Interest aggregation and PIT lifetime management based on input characteristics [16]. SECaaS studies the deployment of security functions at the multi-access edge, including access-control enforcement and anonymization [17]. These works widen the comparison beyond function invocation: resource placement, forwarding state, and deployable security functions are relevant parts of the surrounding system.

Their stated objectives should not be recast as missing NDNF features. Placement feasibility does not establish invocation permission, parameter-aware aggregation does not determine the assigned predecessor in a collaboration, and deploying an access-control service is a different question from authorizing calls to that service. Conversely, NDNF's application-layer checks do not replace these systems' resource-management or forwarding mechanisms. A claim of superiority would require a common workload and matching requirements; their published results are not interchangeable baselines for the experiments in this

proposal.

2.3 Named Computation, Invocation, and Composition

NFN studies named computation and expression evaluation; NFaaS studies execution placement in a named-function environment [6, 7]. RICE provides an invocation handshake with support for authentication and security information [8]. NSC specifies signed invocation Interests validated using an application-provided Trust Schema, and discusses protected input and result exchange [9]. These designs establish relevant invocation and authorization mechanisms. Experimental comparisons below refer to the tested NSC configuration, not every feature of its full design.

For the collaboration extension, the question is how a request gathers current willingness and feasibility information before assigning execution roles, and how that assignment constrains intermediate-object use. Long-term service permission does not identify the Provider assigned to a particular predecessor role. Neither a request ID nor a valid signature alone expresses that assignment. The proposed contribution is the enforced connection between runtime assignment and object use, rather than the manifest format itself. Service-composition work such as CaSCON [18] is a relevant comparator; the research will examine assignment timing, dependency validation, and recovery semantics alongside workflow-security and capability-based designs. Novelty beyond this proposed scope remains to be established by that comparison.

2.4 Comparing Authorization Designs under the Same Requirements

2.4.1 DNMP as a Concrete Trust-Schema Design

DNMP uses role-constrained signing to authorize measurement commands [10]. Its Network Key anchors the signing hierarchy for role keys, device keys, and the distributed schema. Given `target`, `probeType`, `probeArgs`, and the user's `roleType`, the client finds a schema-permitted signing key among the user's available keys. It rejects a missing match

and selects the least privileged key if several match. The receiving Network Observer Daemon (NOD) dispatches a probe only after publication validation under the schema. Thus, this is command authorization, not merely verification of an arbitrary valid signature. DNMP also bounds publication lifetimes and retains recent publication state for replay protection, assuming loosely synchronized clocks.

2.4.2 *Adapting the Alternative to NDNFSF*

For comparison, we propose retaining the NDNFSF Request–ACK–Selection–Response workflow while replacing ABE-backed invocation checks with role-constrained signing rules. The alternative must authorize both User invocation and Provider participation, match messages to active requests, and permit execution only after Selection. These adaptations are not features attributed to DNMP itself. Both designs require descriptor confidentiality before selection, recipient-scoped input/result protection afterward, and checks for withdrawn authority.

Table 2: Alternatives under the same discovery and execution requirements.

Design	Execution decision	Non-public discovery	Management cost
DNMP-inspired alternative	Validate role-constrained signatures; check invocation state and selected Provider	Combine authorization with service-group encryption or ABE for discovery	Role credentials, trust rules, status or expiry checks, and encryption keys
NDNFSF ABE-backed	Check signatures, RequestNonce, ProviderChallenge, current permissions, and request state	Encrypt a descriptor for a service-authorized receiver set without listing current candidates	Identity credentials, ABE policies/DKEYs, status distribution, and rekeying

Role granularity is policy-defined: a role can cover multiple actions, and credentials can be reused where the rules permit. DNMP is therefore a concrete authorization precedent, not evidence that every service needs a new signing certificate.

This comparison concerns complete designs, not measured DNMP-versus-NDNFSF performance. The invocation-state and encryption choices must be implemented and evaluated under matched requirements before claiming a cost advantage.

Recipient encryption protects selected exchanges; service-group encryption or ABE can protect discovery before participants are selected. Trust Schema alone does not provide confidentiality, but it can be combined with either mechanism.

The proposed alternative requires current revocation-status checks or certificate expiry. Deleting a certificate from a repository does not invalidate cached copies. We do not attribute NDNSF's runtime revocation protocol to DNMP.

2.5 Protected Discovery and Permission Management

NDNSF uses ABE for two related purposes: protecting discovery descriptors before Provider selection and managing service permissions independently of identity-signing credentials. The User publishes encrypted descriptors for shared retrieval, while the Controller issues policy keys defining decryption access. The enhanced NAC-ABE library provides KP-ABE support [19]. NDNSF connects these permissions to invocation by checking protected challenge values, message signatures under trust rules, and current authorization/request state. Content encryption supplies confidentiality; the combined checks govern participation and execution. Section 3.4.1 explains the reuse of permission management and its additional costs.

The comparison must include group-key or broadcast-encryption alternatives where appropriate. A stable, small group can make conventional group encryption attractive; known Providers and simple permissions can favor signed invocation with recipient encryption. ABE becomes attractive when recipients are identified by service entitlement before their current identities are known, and multiple entitlements should remain independent of identity-signing credentials. Policy size, membership churn, encryption cost, and authority availability bound this argument.

Chapter 3

Research Approach: Invocation and Collaboration

This chapter first describes the invocation and authorization design under evaluation, then introduces the proposed collaboration extension. Later confidentiality and lifecycle refinements are not attributed to every initial-prototype path. The basic invocation selects one Provider; role assignments and named dependency edges belong to the extension.

3.1 Components and Application Responsibilities

The framework separates application handlers from the machinery that discovers Providers, correlates messages, and checks invocation state. A `ServiceUser` manages outgoing requests and their callbacks. A `ServiceProvider` registers service handlers and processes selected invocations. A `ServiceController` manages permissions and authorization material. These are software roles: a ground station can consume camera services while providing image analysis, and a compute node can request an upstream service while executing its own assigned work.

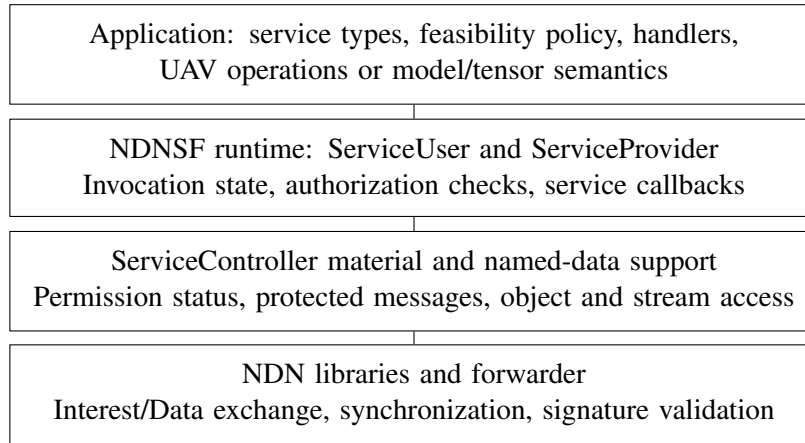


Figure 1: Responsibility groups in the framework. Lines denote software dependencies, not packet directions or a mandatory deployment topology. The Controller is a logical authority, not an on-path relay for every exchange.

Application code defines the meaning of a service and supplies its feasibility and execution logic. The framework cannot determine from a valid ACK whether a camera truly

has an unobstructed view or whether an advertised GPU can meet a prediction deadline. It can associate the ACK with the request, verify the participating identity and required protected values, and apply the configured selection policy. A positive ACK expresses willingness subject to the request's conditions; it is not evidence that execution has finished or a reservation unless the protocol explicitly provides one.

The ServiceContainer supports composition of runtime roles and local services within a process. Its local registry gives trusted components a way to invoke registered application functionality without turning every internal call into a network exchange. The application must keep the runtime and callback dependencies alive until active operations are closed. Local composition is useful for shared camera or inference facilities, but remote callers cannot select it as an alternative authorization path.

The current typed C++ interface illustrates this separation. The following shortened example uses application-defined, serializable status messages; identity provisioning, event-loop setup, and error handling are omitted. It illustrates API use rather than a standalone executable.

Listing 3.1: Typed registration and invocation of a compact status service.

```
provider.addHandler<StatusQuery, StatusReply>(
    ndn::Name("/status"),
    [](const ndn::Name& requester,
        const StatusQuery& query, StatusReply& reply) {
        readApplicationStatus(query, reply);
    });

user.RequestService<StatusQuery, StatusReply>(
    candidates, ndn::Name("/status"), query,
    [](const StatusReply& reply) { displayStatus(reply); },
    []() { reportTimeout(); }, 2000);
```

The handler interprets the query and fills the reply. The User interface supplies an

invocation timeout and result callbacks, while NDNFS manages the surrounding exchange. A service requiring confidential execution input uses the protected input-delivery path described below; this compact example does not prescribe placing arbitrary application input in discovery. Typed serialization removes some repeated application glue, but does not establish semantic compatibility between different service versions. That remains part of the service contract.

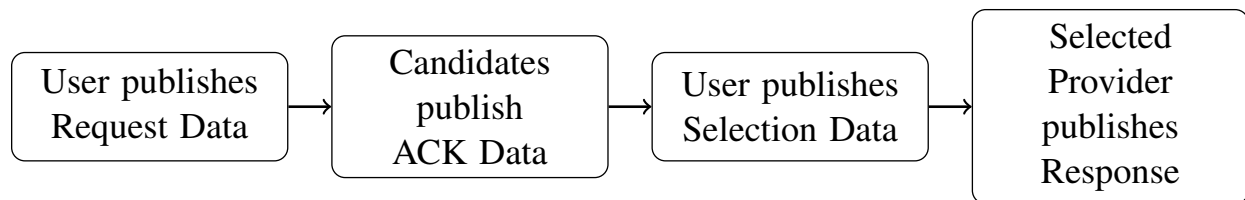
3.2 Participants, Trust, and Exclusions

These are protocol roles, not necessarily separate devices: one node may be both a User and a Provider. The managed organization registers identities and approves service permissions; the Controller administers that policy. Participants require a provisioned trust anchor and validation rules for identity certificates and Controller material. Not knowing which candidates will respond to this invocation does not mean that the authorization authority has never enrolled those entities. Billing, arbitrary business transactions, globally optimal scheduling, and protection against all denial-of-service traffic are outside the claimed scope. The User requests a service and selects its Provider; in the collaboration extension, it also coordinates role assignments. Providers offer and execute services. A Controller administers authorization material and signed status. Forwarders and caches can deliver Data without being trusted to authorize execution. The model trusts the Controller and the User's planning decision; it does not claim consensus against a malicious coordinator. An attacker may observe visible names and traffic, inject invalid objects, replay previously valid messages, or operate without the needed service permission. Credential compromise and intentional sharing by an authorized principal require additional controls and are not prevented by ABE alone.

Authorization is permission to perform an operation, authentication establishes the signing identity under trust rules, and confidentiality limits who can recover Content. Valid credentials do not prove honest load reports, correct inference, or truthful sensor observations. Signed status must be obtained and validated before it can affect local authorization; a disconnected participant cannot be assumed to learn revocation immediately. Deadline/expiry

checks require an explicit clock policy. Tests must report the allowed skew and failure behavior rather than silently assume synchronized clocks.

3.3 Four Named Message Types



Controller: authorization material and signed status.
 Sync: discover names. Interests: retrieve Data.
 User/Provider: validate Data, check permissions
 and request state before acceptance.

Figure 2: Logical invocation dependencies. Arrows express application causality, not unsolicited network Data delivery.

Table 3: Logical message responsibilities; this is not a wire-format listing.

Data	Purpose	Principal checks
Request	Publish service requirements, minimized discovery descriptor, request identity, and RequestNonce	Validate signature and current authorization context; recover the protected descriptor before evaluating willingness
ACK	Report positive or negative participation decision; a positive ACK carries selection information and ProviderChallenge	User checks signer under trust rules, service permissions, request ID and returned RequestNonce; only eligible positive ACKs enter Provider selection
Selection	Notify the chosen Provider and supply protected input or input references and request-scoped keys	Provider validates User signature and Selection using stored ProviderChallenge; checks intended Provider, request state, and service permissions
Response	Publish completion/result metadata or references to protected results	Check authorized producer, request context, result identity, and decryption scope

RequestNonce is the User-generated application-level random value; ProviderChallenge is the Provider-generated challenge checked through Selection. Their protected exchange is explained below. The table describes the single-Provider invocation. Plan identity, role assignments, and dependency checks are added by the collaboration extension below, not by the initial four-message flow.

These are NDNSF Data objects. Sync supports discovery of their names, after which Interests retrieve Data, possibly from a cache. A negative ACK is an application participation decision, not a network Nack. Not every observed Request must receive a reply: invalid, unauthorized, duplicate, or expired messages are subject to validation and state checks. A positive ACK expresses willingness; it does not itself start the selected task or guarantee later resource availability.

3.4 Authorization and Confidentiality

3.4.1 Rationale for ABE-Backed Service Authorization

NDNSF uses ABE to protect discovery descriptors and reuses attribute-based service permissions in its invocation checks. The first choice addresses confidentiality before the participating Providers are known; the second keeps service entitlements independent of identity-signing credentials. Neither implies that ABE is always preferable to role-based authorization with suitable encryption. Withdrawal and rekeying are lifecycle costs to evaluate. Execution authorization determines whether a User may invoke a service; Provider authorization determines whether an entity may offer that service. Content confidentiality limits who can read protected content. The Controller administers these service permissions; the User and Provider check them during invocation. Signature validation under trust rules and invocation-state checks remain necessary. Permission to offer a service is distinct from selection for one particular task.

The design addresses three requirements: keep discovery descriptors non-public before participating Providers are identified; authenticate participants, verify their service permissions, and reject replayed invocation messages; and support permission grants and withdrawals during

operation. Permission aggregation is an additional management benefit, not a fourth security guarantee.

Reason 1: confidential discovery before Provider selection. Before Provider selection, the User publishes a discovery descriptor containing the location, deadline, and resource requirements needed for a Provider's positive-ACK decision. For example, nearby UAVs need an observation area and arrival deadline to decide whether to participate, but do not yet need the full application input. The User encrypts this descriptor using ABE and the requested service's attribute, without enumerating Provider identities. Confidentiality is against parties lacking the required decryption capability: authorized but unselected Providers can still read the descriptor. Content encryption does not hide outer Data names or traffic patterns. In the request-scoped confidentiality path, the full application input is encrypted with a request-scoped key; Selection protects that key for the selected Provider. The requesting User retains access. A Trust Schema alone does not encrypt content; a role-authorized design can also use ABE or service-group encryption to provide discovery confidentiality.

Encrypted Request Data can be retrieved or cached without granting decryption access. This separation of delivery from disclosure also holds for other content-encryption schemes; it is not unique to ABE.

Reason 2: reusing attribute-based permissions for service invocation. Confidential discovery alone does not justify ABE-backed execution authorization: ABE encryption could instead be combined with role-based invocation checks. NDNSF chooses to administer both Provider and User service permissions through the existing attribute-authority infrastructure, separately from identity-signing credentials.

Provider permissions determine access to RequestNonce; User permissions determine access to ProviderChallenge. NDNSF checks RequestNonce in ACK Data and validates Selection Data using the stored ProviderChallenge. Signature, current-permission, and request-state checks remain separate requirements (Section 3.4.2). Decryption is therefore one part of the invocation

check, not permission to execute any request.

DNMP instead uses role signing keys and a Trust Schema to constrain command targets and actions [10]. In the DNMP-inspired alternative (Section 2.4.2), retaining ABE for discovery would require managing both decryption permissions and role-based invocation permissions. NDNSF uses the attribute-authority infrastructure for both service-level checks, without adding role signing credentials solely to represent service membership. Trust rules for validating Data signatures remain necessary.

This is a management-reuse rationale, not a demonstrated cost advantage. Provider and User permissions remain distinct; discovery encryption does not already supply User permission decryption keys or challenge processing. For policies requiring command-argument restrictions, DNMP's schema-based checks may be a better fit than NDNSF's service-level checks. The evaluation must compare both designs at the same policy granularity, including credential updates and invocation overhead.

Reason 3: service-permission aggregation as an additional management benefit. KP-ABE associates attributes with ciphertext and an access policy with each decryption key (DKEY) [19]. NDNSF uses `/SERVICE/<service>` for Provider access and `/PERMISSION/<service>` for User access. Within one authority and parameter generation, an entity's service permissions can be combined in one individually issued DKEY. For example, one User's policy can allow detection OR mapping, while another allows detection only. Their identity-signing credentials need not change to represent these different service sets; each service retains its own ciphertext attribute.

Compared with DNMP's role-key-based authorization [10], NDNSF represents service entitlements in the DKEY rather than in additional role-specific signing credentials. Where equivalent entitlements require several distinct role credentials, this avoids issuing and maintaining those additional signing credentials. It does not imply one credential per DNMP service: one role can permit several commands. Nor does it establish a universal advantage over

Trust Schemas, which can express other credential arrangements. A single DKEY does not have fixed size or update cost; policy complexity, distribution, and rekeying still matter. The benefit concerns independently managed credentials, not a demonstrated reduction in total key bytes or runtime cost.

3.4.2 *Authorization Mechanisms, Lifecycle, and Costs*

Identity authentication. The User and Provider validate Data signatures and certificate chains under the applicable trust rules. This identifies an accepted signer; it does not alone establish that signer’s permission to offer or invoke the requested service.

Service-permission verification. In the ABE-backed invocation path, the User generates a fresh random value, *RequestNonce*, and includes it in ABE-protected Request content. This is an NDNSF application-level value, not the NDN Interest Nonce. A Provider that recovers it returns it in ACK Data; the User compares the returned value with the one stored for the active request. Thus *RequestNonce* supports both request binding and a check of access to the service-protected value. The Provider generates *ProviderChallenge* and includes it in ABE-protected ACK content. The User uses the recovered *ProviderChallenge* to construct Selection Data. Before execution, the Provider validates Selection Data using *ProviderChallenge* stored for that request. Together with current service-permission checks, these checks determine whether the Provider may offer the service and whether the User may invoke it.

These checks demonstrate access to protected challenge values, not the creation of new permissions or a formal proof of exclusive key possession. Authorized key or challenge sharing is outside the claimed threat protection. This reuses attribute-based permission management rather than a separate set of role-based invocation rules. It is not cost-free: confidential discovery already needs Provider DKEYs, whereas User permission DKEYs and challenge processing are additional authorization costs. ABE confidentiality combined with DNMP-inspired role authorization remains a valid alternative.

Replay protection. Participants match messages to an active request and its protocol state. The Provider records whether `ProviderChallenge` has already been used to accept Selection Data for execution; receiving the same Selection again must not start another execution. This requires appropriate retention and invalidation of request state, including across restart. Neither encryption nor possession of a challenge value alone prevents replay. These admission checks do not establish exactly-once application side effects across crashes.

Runtime permission grants and withdrawals. The Controller updates permission status for both operations. An ordinary grant updates the target entity's complete permission policy; that entity retrieves a replacement DKEY without changing the ABE public parameters. Withdrawal additionally rotates the Controller's ABE master key and public parameters. Entities retaining permissions obtain replacement DKEYs encoding their remaining permissions. The master key stays with the Controller. Rotation is Controller-wide, not confined to the withdrawn service.

In the revised dissemination design, the Controller publishes updated public parameters as signed named Data and uses Sync to announce their availability. Participants retrieve and validate the parameters and Controller-signed `PolicyStatusData`, then refresh their DKEYs as required. Ordinary grants also require participants to discover and retrieve updated permission status, even though the public parameters have not changed. Sync is an update-discovery mechanism, not the authority for granting permission. The local implementation supports status retrieval and parameter rotation; the dedicated Sync announcement path remains to be verified end to end.

Current-permission checks enforce withdrawal after the updated status has been retrieved and validated. Parameter rotation restricts exchanges encrypted under the new generation. Disconnected nodes cannot apply updates immediately, and previously disclosed keys or plaintext cannot be taken back.

Lifecycle comparison and evaluation. DNMP specifies validity bounds and replay-state handling [10]; these do not by themselves provide immediate withdrawal of an unexpired role

key. For the comparison, a DNMP-inspired design can revoke the relevant credential and require execution nodes to check signed revocation status or reject expired certificates. This invalidates authority at the validator; it does not physically retrieve distributed certificates. Both approaches require update propagation, and neither retracts previously disclosed keys or plaintext.

Focused integration tests exercise NDNSF's runtime revocation checks, including identity-, certificate-, and service-scoped withdrawal. These tests establish behavior at the exercised runtime boundaries, not network-wide instantaneous revocation or lower cost than certificate-based alternatives. The remaining evaluation should measure status-propagation delay, rejection delay, rekeying traffic, and effects on unaffected members. Runtime status validation and key rotation support runtime permission changes, with explicit propagation and rekeying costs. They are not ABE-exclusive capabilities. NDNSF introduces ABE for confidential discovery and reuses its service-permission management for invocation checks and permission updates; signatures and request-state checks remain necessary, separate parts of the design.

Discovery confidentiality applies to descriptor Content against parties lacking the required ABE decryption capability. It does not conceal the descriptor from other authorized candidates or provide traffic-analysis resistance. In the request-scoped protection design, the User encrypts application input with a symmetric key and delivers the necessary key material through Selection, encrypted for the chosen Provider's authenticated public key. The response key is shared by the requesting User and selected Provider. Other Users cannot decrypt the protected result under the key-secrecy assumptions; the Provider necessarily knows the plaintext it produces. These are the protection requirements for the evaluated design, rather than claims about every historical implementation path.

3.5 Invocation Modes and Execution Lifecycle

3.5.1 Discovery, Known Providers, and Local Composition

Normal discovery is appropriate when the executor has not been chosen. The User collects ACKs within a bounded window and chooses among positive ACKs that pass the

applicable validation and policy checks. A negative ACK communicates non-participation; silence cannot distinguish unwillingness, loss, and disconnection. A selection strategy chooses among candidates for the requested operation. Choosing several independent executions, where supported, is distinct from assigning complementary roles with dependencies in a collaboration plan.

Known-Provider invocation is appropriate when application meaning fixes the device, for example, stopping the camera that was previously started. The current `RequestServiceTargeted` path can reuse previously bootstrapped one-time authorization material to avoid repeated ACK/Selection exchanges under its supported conditions. It still checks current authorization and request state. The fast path is not universal: request-scoped confidentiality needs a Selection envelope to carry the input and response keys, and the current implementation uses the bootstrap/ACK/Selection path for that case. The explicitly Selection-gated input mode is rejected by the Selection-free Targeted interface. Thus, targeting a known Provider is an application requirement, whereas omitting exchanges is a conditional implementation optimization.

Trusted local composition operates within a process through the local registry. It has neither remote discovery nor the same network threat boundary. Evaluation should identify whether a handler was reached locally, by normal discovery, or by a targeted exchange. Otherwise an apparent invocation improvement might merely measure a different operation.

3.5.2 Admission, Progress, and Failure

An API submission, admission to a runtime queue, acceptance by a Provider, and completion of application work are different events. Under overload, reporting latency only for completed requests can hide dropped or rejected work. The runtime therefore needs observable outcomes for refusal, timeout, and terminal results, while an evaluation must count all submitted requests in its stated denominator. Selection-status queries can help determine the observed status of selected work; they are not proof of a physical effect or a distributed transaction.

Applications must also distinguish cancellation of interest in a result from reversal of an action. A recording can be stopped, but previously recorded frames still exist. An inference worker can stop producing new output, but a completed remote action cannot be undone by deleting requester state. Retrying requires the application's stated idempotency or compensation policy. For the proposed collaboration extension, replacement additionally requires old assignments and late callbacks to stop advancing the current plan.

3.6 Control Messages, Streams, and Complete Objects

Small service messages communicate intent, permission evidence, and status. Large application data should not be copied into each control message. An object reference allows a service to identify a model fragment, recording, or image, while the recipient retrieves its named Data separately. The reference and service contract must supply enough information to select the intended version and validate the reconstructed object. The concrete interface determines the available name, size, digest, segmentation, and protection metadata; not every object type needs an identical collection of fields.

The important separation is between identifying an object, obtaining its bytes, and authorizing its use. A digest can detect a mismatch with the expected bytes only when the expected digest itself is trusted. A signature binds Data to a key under the application's trust rules. The invocation or collaboration checks establish whether that producer and object may satisfy the current input. Recipient-scoped key delivery controls access to protected Content. None of these checks should silently substitute for another.

An ongoing stream has a different lifecycle from a complete object. Live video may prefer recent frames and tolerate explicitly identified gaps. A model artifact generally requires every needed byte before loading. Incremental inference requires ordered application events and a terminal outcome, but its token stream is not necessarily interchangeable with a camera stream interface. NDNFSF supplies named-data mechanisms; applications define acceptable gaps, buffering limits, and completeness rules. The evaluation must measure the chosen policy,

including its discarded data and queueing costs.

Named-object reuse is valuable in the multi-view case: the compute Provider returns annotations and references to the original images, with authorized access to their content keys. The requester can retrieve those images from available producers or caches and validate their original signatures. This avoids producing a second image object merely to change the receiver. It does not guarantee that a cached copy remains available; retention requirements need an explicit producer or repository policy.

3.7 Proposed Extension: From ACKs to a Collaboration Plan

The extension changes the selection problem from choosing one executor to assigning complementary task roles. The application supplies a role/dependency template, required services, feasibility constraints, and deadline. The template may be written manually; automatic generation is not necessary to establish the framework contribution. A replaceable planning policy selects among eligible positive ACKs after the collection window closes. Closing the window freezes the candidates observed for that decision, not all possible Providers in the network. ACK claims can become stale, so a selected Provider still performs execution-time admission and may reject work it can no longer execute.

In the proposed design, the User signs a Plan Manifest containing the request identity, accepted candidate snapshot, exact role-to-Provider assignments, permitted named dependency edges, authorization/key scope, and execution identity. Its digest is computed over a canonical plan body that excludes the digest field itself. Providers validate that signed plan before deriving local dependency checks; a long-term certificate does not specify this task's assignments. The User must record the committed plan and reject a different plan for the same execution identity. This is a local state-machine rule under the trusted-User model, not a consequence of signing alone or protection against a Byzantine User signing two plans.

3.8 Proposed Dependency Admission and Execution State

Under the proposed acceptance rules, a downstream Provider checks that the execution is active and within its deadline before accepting a dependency. The Data signature must validate under the applicable trust rules; the request, plan, and execution identity must match the assignment; the signer must be the assigned predecessor; and the name, data format, and key scope must match a declared edge. Required segments must validate, and incomplete or malformed objects cannot satisfy an edge. The same checks apply to producer-, cache-, and repository-fetched Data. The plan references objects in their actual producers' authorized namespaces rather than republishing them under a different producer identity.

Table 4: Proposed task-state contract and required negative checks.

State/event	Permitted transition	Required rejection or bounded behavior
Collecting ACKs	Close window and validate candidate set	Late ACK does not silently alter a committed plan
Selected	Validate Selection and perform local admission	Positive ACK alone, wrong Provider, or spent challenge cannot start work
Waiting for dependencies	Execute when required authorized edges are satisfied	Wrong role, undeclared edge, incomplete object, or wrong active execution is rejected
Completed/cancelled	Return recorded status under duplicate-message policy	Duplicate publication does not implicitly repeat a side effect
Failure/reassignment	Explicitly terminate or authorize a replacement execution	Old execution must not acquire authority from the new assignment; allowed result reuse is explicit

These are proposed acceptance conditions, not a claim that every production path is qualified. Network re-retrieval is not identical to repeating computation. If a task is reassigned, cancellation, fencing, and result-reuse rules must be explicit. An already computed image may remain useful when the new plan explicitly authorizes its reuse; an old object is not automatically invalid solely because it is cached. Application-defined compensation can limit side effects but does not provide distributed atomic transactions or exactly-once physical action.

The authorization design and comparative evaluation are intended to answer RQ1; the proposed plan/state rules and their tests are intended to answer RQ2. Neither question is answered by a design description alone. Existing invocation evidence does not establish collaboration correctness; wrong-role, wrong-edge, replay, cancellation, and direct/cache tests remain separate evaluation obligations.

3.9 Conditional Correctness Argument

Assume uncompromised signing keys, correctly enforced trust/authorization checks, and Providers that obey the specified state machine. Selection-before-execution follows only if every handler-start path checks an active assignment and unspent authorization state. Dependency integrity follows only if every input path, including a cache hit, checks the assigned producer and declared edge before satisfying the dependency. Replacement isolation additionally requires old workers and late callbacks to lose authority under the new execution identity. Restart recovery must preserve or safely invalidate the relevant one-time state; signature validity and cache FreshnessPeriod alone are insufficient.

These are conditional invariants and implementation proof obligations, not a cryptographic proof of a complete system. Tests should inject a wrong-role object within the same request, an undeclared edge from a correctly signed producer, and a late callback after replacement. Each isolates a property that a request ID alone cannot express. Application result correctness and physical side effects remain outside this argument.

Chapter 4

Application Scenarios and Validation Plan

4.1 Validation Strategy: Applications Exercise the Framework

We use NDNFSF-UAV and NDNFSF-DI to evaluate how NDNFSF supports service discovery, authorization, data delivery, and execution management across both single-Provider and multi-Provider workflows. Single-Provider operations remain relevant to RQ1 and RQ3; workflows with complementary roles additionally examine RQ2. The multi-view and distributed-model cases below are representative workloads within these broader applications.

The evaluation maps each application function to the NDNFSF path it actually uses and to an observable outcome. An application feature supplies evidence only when that path is exercised and the result is recorded. A local helper call, for example, does not establish remote invocation authorization. Likewise, a working interface or model does not by itself demonstrate framework correctness or performance. The chapter defines validation coverage; Chapter 5 distinguishes available measurements from remaining tests.

4.2 NDNFSF-UAV: Control, Sensing, and Collaboration

4.2.1 Application Structure and Service Workflows

The UAV application connects an operator’s ground station with drone-side service containers. Drone services expose camera operations, telemetry, and mission-related operations through NDNFSF. The ground station can also provide an object-detection service, so the same application includes both service consumption and service provision. Application code interprets camera, vehicle, and mission data; NDNFSF supplies the reusable invocation and data-delivery mechanisms.

These operations cover different framework requirements. Selecting a currently available sensing Provider exercises discovery and ACK-based selection. Addressing an already chosen vehicle exercises invocation associated with that Provider rather than selecting an arbitrary

substitute. Telemetry and mission-status retrieval exercise the association of returned Data with the intended vehicle and active operation. Long-running operations supply cases for cancellation, deadlines, and handling late status. Tests must distinguish rejection of an invalid command from the flight controller's physical execution of an accepted command.

Live video and recordings exercise different data paths. Camera Start/Stop is a service-control operation; the resulting live stream uses the named-data streaming interface. Recording references and complete stored objects require exact object retrieval rather than assumptions about cache persistence. These functions therefore contribute to validating stream lifecycle, ordering and gap handling, protected content delivery, and data-product retrieval. Video codecs, image processing, operator interfaces, and flight safety remain application responsibilities; their use of NDNFS is nevertheless part of framework validation.

4.2.2 Operator Workflow and State Interpretation

A typical session starts with an operator discovering sensing services and choosing a vehicle. Subsequent camera or mission operations retain that vehicle identity unless the operator or a stated mission policy authorizes replacement. The interface associates status with the vehicle and operation that produced it. A delayed observation must not be presented as evidence that a newly issued command succeeded. Observation time, operation identity, and the application's stale-data threshold are therefore part of the validation contract.

The proposed evaluation follows a complete session rather than testing isolated buttons: obtain status, start a camera stream, inspect incremental frames, stop the stream, find a recording, and retrieve the selected recorded object. Camera control uses service invocation; frame delivery and recording retrieval use their respective data paths. An implementation may record while no live viewer is attached, so recording availability should not be inferred from stream subscription. The test records whether the configured application supports that mode and whether recording references identify the correct producer and version.

Flight-related services connect the application to the vehicle's flight-control interface,

Table 5: UAV workflow contracts to exercise in application validation.

Operation	Application meaning	Observable framework outcome
Telemetry and status	Observation of the selected vehicle	Correct producer and operation association; stale data identified
Camera Start/Stop	Change the selected camera’s activity	Authorized invocation and terminal status distinct from frame arrival
Live video	Recent frames from the requested stream	Start delay, useful frame delivery, gaps, bounded buffering
Recording retrieval	Obtain a particular stored capture	Exact object and producer validated; Content access enforced
Mission operation	Issue bounded work and follow progress	Admission, progress, timeout, and completion distinguished

such as MAVLink and PX4 [20, 21]. Authorization to submit a command is separate from its operational safety. Geofencing, vehicle readiness, collision avoidance, and local failsafe behavior belong to the vehicle/application safety design. The networking study should use bounded supervisory commands and an explicit loss-of-contact policy, rather than assume that a service invocation is suitable for every low-level flight-control loop.

Multi-vehicle patrol provides a secondary workload: an application divides a region into tasks, tracks their completion, and decides whether unfinished tasks can be reassigned. Its relevance is the association of task state and returned observations with the active assignment. This is broader than multi-view analysis, but it should not become an additional unbounded algorithm-development requirement. The primary collaboration experiment below fixes its task and analysis method so that network and protocol effects can be isolated.

For both workflows, failure experiments should include a missing observation, a disconnected selected UAV, and delayed status after a replacement decision. Report the time during which a feasible Provider is actually reachable as well as the request deadline. Retransmission and synchronization cannot manufacture radio coverage or extend an expired application time budget. A partial completion rate can therefore reflect availability limits even

when retrieval recovery behaves correctly.

4.2.3 Representative Collaboration: Multi-View Collection and Analysis

An operator chooses a target location or area on a map. Request Data carries only the service-protected descriptor needed to assess coverage, viewing feasibility, sensors, and timing. Eligible UAVs return positive ACKs. The proposed workflow assigns complementary viewing roles and a processing role, then delivers execution details and appropriate keys through Selection.

Each sensing UAV signs encrypted ViewImage Data under its own authorized namespace, with observation time and available pose/calibration metadata. The compute Provider retrieves permitted images, performs the specified analysis, and publishes signed Annotation Data. Its encrypted result supplies original image references and permitted image keys to the requester, which retrieves the images and renders annotations. The workflow reuses the original signed images rather than republishing them under another producer's identity.

This case examines role assignments, named dependencies, result reuse, and incomplete-view or processing failures. It does not replace the control, telemetry, video, and recording validation above. GPS alone does not establish camera visibility, depth, or correspondence between views. Improved recognition or localization requires ground truth, a single-view control, and a specified fusion method. MiniNDN [22] and PX4 SITL [21] can exercise network behavior and application/autopilot integration respectively; neither establishes real-flight safety.

4.3 NDNSF-DI: Inference Lifecycle, State, and Collaboration

4.3.1 Application Structure and Inference Workflows

NDNSF-DI provides an inference application layer over the framework. It interprets model and task descriptions, coordinates model preparation and execution, and exposes inference results to callers. Its native requester and Provider interfaces include request submission, service

registration, result/event handling, and cancellation; conversation-related interfaces also carry continuation state. The presence of an interface is not evidence that every model or recovery configuration has passed end-to-end tests.

A single-Provider inference request exercises service discovery, permission checks, selection, protected input delivery, and result retrieval without requiring model partitioning. Preparing a cold model and reusing a resident model exercise different readiness and artifact-access paths. Model files and recorded artifacts may use managed repository storage; the forwarding Content Store remains opportunistic caching, not guaranteed persistence. DI defines model identity, compatibility, and placement policy, while NDNSF supplies service lifecycle and named-object exchange.

Streamed inference and conversation workflows extend coverage to incremental outputs, cancellation, terminal status, and request-specific state. Model weights, activation tensors, and key/value (K/V) state have distinct lifetimes. The model computes K/V tensors and the inference runtime retains them for later decoding; shared buffers can avoid copying the whole cache at each step [23]. Resident weights do not replace this state. DI owns tokenization and KV compatibility rules; framework validation concerns the authorized request, data exchange, and lifecycle through which the application uses that state. A KV speedup is not automatically an NDNSF speedup.

4.3.2 Representative Collaboration: Multi-Provider Inference

A planner combines a supported model description with runtime ACK capabilities, assigns Providers, and prepares their model fragments. Pipeline partitioning assigns sequential layers or layer groups. Tensor partitioning divides operations or tensors and requires the corresponding collective exchanges. Providers exchange activations or collective inputs over declared dependencies. ONNX [24] describes the computation graph; export alone does not implement cross-Provider collectives or establish numerical correctness. Model partitioning has substantial prior work [25, 26]; the NDNSF question concerns runtime assignment and authorized named-data execution.

Autoregressive generation also uses each generated token in a later decoding step. This temporal dependency differs from forward computation through the layers. The evaluation separates model preparation, prefill, time to first token, subsequent decoding, state storage, and data transfer. Claims remain specific to tested models, precisions, partitions, and devices. A feasible single-Provider baseline is retained; if a model cannot fit one device, distributed feasibility is meaningful but a speedup against a non-running baseline is not.

4.3.3 *From a Model Graph to Runtime Assignments*

Model preparation and Provider assignment occur at different times. Preparation identifies a supported model, its tensor interfaces, and candidate partitions. It can produce a reusable template describing fragments and their dependencies before any particular invocation. Runtime planning then uses the request and collected positive ACKs to bind those roles to actual Providers. A manually authored template is compatible with this design: automatic partition search is not required to test the protocol. The Controller’s authorization role does not make it the model partitioner.

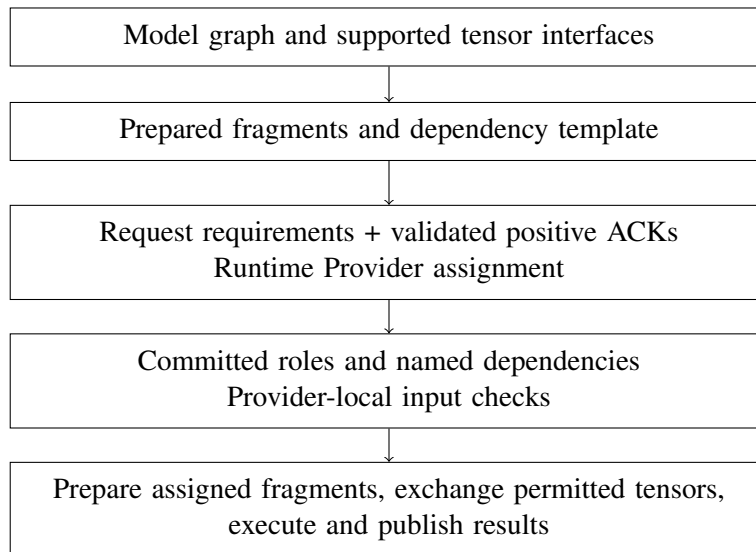


Figure 3: Proposed separation of model preparation and request-specific collaboration. Arrows show processing order, not a claim that every model graph or assignment strategy is implemented.

Each fragment contract needs defined input and output tensor names, shapes or supported dynamic-shape constraints, element types, serialization, and model-version compatibility. A split across a skip connection or a branching graph may have multiple live outputs, not a single vector. The receiving fragment must obtain all required inputs before execution. A useful negative test supplies a correctly signed tensor with the wrong shape or model identity: protocol provenance may be valid while the application contract is not.

Pipeline partitioning assigns consecutive computation stages and transfers their boundary tensors. Tensor parallelism splits an operation across participants and adds exchanges such as reductions or gathers. Model replication assigns independent work to complete copies. These are different execution strategies, not interchangeable names for placing a model on several devices. Early exit changes how much model computation is performed and must be evaluated separately from placement. ONNX export supplies a graph representation [24]; usable cuts and collective operations still depend on the supported runtime and model operators.

Preparation cost matters when a Provider lacks the required artifact or compatible loaded session. Evaluation should distinguish artifact transfer, session creation, accelerator memory allocation, and execution. A warm run that reuses resident weights cannot be compared to a cold baseline without reporting that difference. Placement can favor a Provider that already holds compatible state, but that policy must respect request authorization and cannot assume that cached state is transferable to any new request.

4.3.4 Tensor Exchange and Autoregressive State

For a pipeline, an upstream fragment produces an activation tensor, makes it available as protected named Data, and supplies its reference through the declared dependency. The downstream fragment retrieves and validates that object, checks the tensor contract, and performs its assigned computation. A fan-out reuses an upstream output for several authorized consumers. A fan-in waits for the set of inputs required by the computation. The collaboration protocol decides which producers and edges are permitted; DI decides whether the tensor values and

layouts are suitable for the model.

Autoregressive inference adds a dependency across decoding steps. When K/V caching is enabled, computation after prefill uses the preceding token and retained K/V state instead of recomputing the full prefix. With layer-based partitioning, each participating runtime can retain the K/V tensors for its assigned layers. A generated token or the information needed to select it returns to the component driving the next decoding step. Consequently, the forward path within one evaluation can follow a directed acyclic graph while the whole generation process repeatedly traverses that graph over time.

Keeping weights in accelerator memory does not eliminate the logical K/V inputs. A runtime may reuse device buffers and bind them directly rather than serialize and transmit the entire cache on each step [23]. Reassignment or conversation continuation needs an explicit compatibility and state-transfer policy; silently starting a replacement with unrelated K/V state is not valid recovery. The evaluation separates savings from local K/V reuse, remote transfer costs, and framework orchestration costs.

The initial numerical oracle should compare a supported partitioned execution with an unsplit execution of the same model, precision, inputs, and decoding policy. Appropriate tensor tolerances or deterministic output comparisons must be specified before measurement. A small synthetic model can validate a tensor route, but cannot establish memory feasibility, throughput, or recovery for a large pretrained model. These controls make it possible to distinguish a working communication path from a correct and useful distributed-inference application.

4.4 Coverage, Evidence, and Completion Boundaries

Table 6 groups the applications by the framework mechanisms they exercise. It is a coverage plan, not a claim that every listed path is qualified.

Each evaluated row must identify the concrete application function, production NDNSF API path, configuration, test oracle, and durable evidence. Source implementation, focused tests, multi-process tests, and measured application outcomes are separate evidence levels. Local calls

Table 6: Application workflows and intended NDNSF validation coverage.

Application workflow	Framework mechanism	Required observation
UAV control/telemetry; DI single-Provider inference	Discovery, selection, service authorization	Correct Provider/request association; unauthorized invocation rejected
UAV live video; DI incremental output	Stream or invocation-output lifecycle	Validated outputs, termination, cancellation, and defined gap handling
UAV recordings; DI model/input objects	Named-object references and retrieval	Exact permitted object reconstructed; tampered or wrong object rejected
UAV mission status; DI preparation/conversation	Operation and request lifecycle	Correct status ordering; late actions cannot revive closed work
UAV multi-view; DI partitioned inference	Role assignment and named dependencies	Assigned producers only; missing dependencies and replacement handled
Permission changes in either application	Controller status and invocation checks	Grant discovered/applied; withdrawn permission rejected after update

cannot substitute for network tests, and a successful service exchange cannot establish correct image analysis, numerical output, or safe flight.

The current native implementation has focused and selected cross-process evidence, but the claimed model, device, and recovery configurations are not all qualified. Historical small-model measurements do not validate newer tensor/hybrid, multi-accelerator, or large-language-model recovery paths. The final campaign should cover the claimed mechanism categories using representative application workflows; it need not turn every optional interface or application feature into a separate dissertation completion gate. The broader application scope is preserved while each research conclusion remains tied to the evidence actually obtained.

Chapter 5

Preliminary Evidence and Proposed Evaluation

5.1 Measured Findings Are Configuration-Specific

Invocation campaigns below measure discovery, selection, and execution under their recorded configurations; they do not test a multi-role collaboration plan. The earlier DI diagnostic is separate application evidence, not proof that the initial framework supported role-aware collaboration.

The following numbers are retained from recorded campaigns, not new experiments in this revision [27]. The independent unit for comparative uncertainty is the seed/run, not each correlated request. Success-conditioned latency must be read with completion and timeout rates. A baseline label identifies the evaluated harness configuration, not every feature of the framework named in that label.

Table 7: Selected evidence and limits; values are unchanged from their records.

Campaign	Observation	Supported interpretation / limit
Ten-seed switching holdout	1,312 matched switching requests; NDNSF minus gRPC seed-p95 difference: -597 ms, 95% CI $[-992, -200]$; versus NSC: -2618 ms, CI $[-2916, -2315]$	Lower conditional tail latency in the tested four-Provider configuration, not universal mobility or success-rate superiority
Six-seed work-efficiency repeat	NDNSF: 1,798/1,800 completed, 1,800 execution starts; gRPC-PAR-4: 1,800/1,800, 7,180 starts	Fewer starts than this parallel policy; starts are not messages, bytes, CPU time, or energy
Earlier small-model DI diagnostic	Ten successful runs/layout: mean 259.4 ms distributed versus 186.8 ms single Provider; p95 283.9 versus 206.3 ms	Splitting was slower for this model/configuration; small success-conditioned sample does not establish a general tail-latency result

In the work-efficiency comparison, NSC-4 also completed 1,800/1,800 requests, with 1.002 execution starts per request (rounded). This does not establish a work-efficiency advantage over NSC. The mobility comparison uses common traces, publication times, deadlines,

and four Providers per system. The switching holdout uses seeds 72–81 and a registered reachability-based classification. Of its 1,312 aligned requests per system, NDNF completed 1,311, gRPC 1,297, and NSC 1,296. Each seed-p95 is calculated from that system’s successful requests, so the latency comparison is affected by different failure sets. Overall coverage controls using seeds 62–71 are a separate campaign, not the parent sample of this holdout. The small-model diagnostic is retained from earlier records; its raw-run/model provenance remains to be rechecked. Communication/setup overhead is a possible explanation, not an established cause.

The transition experiment reports 357/357 completions for NDNF without pre-registering Provider addresses, 0/357 for static-list baselines, and 357/357 for pre-registered baseline variants. It isolates the discovery configuration. A gRPC resolver or other discovery integration can update a candidate list [28]; the result does not prove that gRPC or NSC cannot discover a new Provider.

Historical v0.1 loss results include 599/600 completions at 1% per-link loss, 598/600 at 3%, and a best recorded 539/600 at 10%. The last is a selected historical run, not a current aggregate estimate or a matched comparison against later baseline data. The expression $1 - (1 - \ell)^4$ assumes four independent packet traversals with equal loss ℓ ; it is not the full service-failure probability after retries. Timeouts under heavy loss do not, without further interventions, identify one exclusive cause or rule out protocol/policy effects.

5.2 Evidence Needed for Each Research Question

Table 8: Claim-driven evaluation plan, not a completed-results checklist.

RQ	Comparison	Correctness / failure observation	Costs and outcomes
RQ1	ABE-backed authorization; DNMP-inspired role rules plus content encryption (Section 2.4.2); suitable group-key combinations	Unauthorized call never starts handler; replay, wrong challenge, grants, withdrawn/expired permissions, status delay	Completion, discovery delay, crypto work, credential/policy size, grant and withdrawal traffic
RQ2	Same task graph with static/dynamic assignment; controlled removal of each admission check	Wrong role, edge, name, key or execution; direct/cache parity; incomplete objects; cancellation or reassignment	End-to-end completion, validation cost, intermediate bytes, redundant work, bounded recovery
RQ3	Matched Provider counts, tasks, capacities, deadlines and failure schedules; feasible single-device control	Include failed and timed-out requests; distinguish network, admission, execution and result-retrieval failures	Completion and latency together; cold/warm costs; per-run uncertainty; conditions with no benefit

Each campaign must preserve its source/dependency/model identity, configurations, seeds, measurement start/end points, raw-run references, and analysis scripts. Scaling dimensions should follow the actual claim: candidates and services for discovery/authorization, edges and object sizes for collaboration. Ablation diagnoses a mechanism; an intentionally insecure variant is not a recommended deployment. A startup or collector failure must be diagnosed before attributing it to the harness or protocol. Preserve failed attempts; do not count an experiment that never reached its measurement boundary as a measured protocol failure rate.

5.3 Application Experiments and Interpretation

The experimental unit is a stated invocation or application session, with a recorded workload, software revision, and network configuration. Report submitted, admitted, selected, completed, rejected, timed-out, and cancelled work separately where the path exposes those outcomes. Success rate uses an explicit denominator and deadline. Latency distributions should

identify which requests they include, so faster completed requests do not hide a larger failure fraction. Repeated trials or independent seeds support variability estimates; a single exploratory run is reported as such.

For UAV sessions, record command completion latency, video start delay, useful frame delivery, recording retrieval completion, and status age at the operator. For multi-view work, additionally record the number of usable views, processing completion, and the analysis metric against labeled observations. The framework experiment first holds the recognition algorithm fixed. A separate application study is needed before attributing higher recognition accuracy to additional views.

For DI, report model preparation, prefill, time to first token, subsequent output rate, total completion, peak device memory, and bytes transferred between Providers. Compare cold and warm conditions separately. Evaluate a feasible single-Provider case alongside a supported partitioned case. If the distributed deployment exists because the model cannot fit one device, report that capacity benefit directly rather than inventing a speedup over an infeasible execution. Profile communication, serialization, cryptography, and model computation to locate the limiting cost.

Authorization evaluation should exercise both directions of service permission, not only successful decryption. Negative cases include an unpermitted caller, an unpermitted Provider, a validly signed message with the wrong protected value, and reuse after request state is closed. Grant experiments measure how an enrolled entity learns and applies a new permission. Withdrawal experiments measure update propagation, status validation, parameter/DKEY refresh, and rejection after the update is applied. Disconnected nodes and old ciphertext are separate cases with the limitations stated in Section 3.4.1.

The collaboration ablations retain request-ID isolation and signature validation. Removing assigned-producer checks tests a wrong-role object within the same request; removing edge checks tests an undeclared input from an otherwise authorized participant. Replacement tests inject old work after reassignment. This isolates the extension's contribution instead

of claiming to solve cross-request confusion that the invocation foundation already handles. Correctness tests require the expected rejection and absence of prohibited handler execution, not merely an error message in a log.

Framework reuse can be assessed by documenting which invocation, authorization, object-transfer, and lifecycle components are shared by UAV and DI, and which adapters remain application-specific. Source size alone is not a sufficient maintainability metric. Record the changes needed to add a service type and the contracts that must be supplied. This supports the framework's engineering rationale while leaving protocol correctness and performance to their own evidence.

Deployment records should include dependency versions, model hashes, topology, link parameters, clocks, retry policies, cache configuration, and key-management settings without publishing secrets. Separate application failures from startup, configuration, and collection failures. A failed setup is not evidence of a protocol's inability to complete a request. These records allow the final dissertation to distinguish supported claims, useful negative results, and configurations that remain untested.

Chapter 6

Research Plan, Feasibility, and Risks

6.1 Core Research Scope and Feasibility

The immediate priority is a frozen, runnable baseline for the final claims, not completion of every development feature. Select one UAV collection–processing graph and one supported DI partition first. Keep pipeline/tensor/hybrid generality, LLM streaming, and sophisticated recovery only where the implementation and evidence can support them. A negative performance result is an acceptable research finding when its conditions and costs are explained.

Table 9: Revised internal targets; defense is conditional on evidence and committee approval.

Period	Research milestone / decision
September 2026	Agree on three RQs and bounded claims; freeze supported application graphs and implementation/evidence baseline; resolve correctness blockers
October 2026	Execute registered correctness and matched comparison cases; reduce optional scope if an end-to-end gate remains open
November 2026	Freeze accepted results and provenance; write RQ answers, costs, and limitations; keep missing results visibly unverified
December 2026	Deliver an integrated dissertation draft to the advisor; prioritize analysis and revision over optional features
January 2027	Advisor/committee review, revisions, defense readiness decision, and confirmation of current university submission/notice requirements
February 2027	Target final dissertation defense after required reviews and evidence closure; allow time for substantive corrections
Spring 2027	Complete approved corrections and final submission by the applicable Graduate School deadlines; confirm exact dates separately

The February 2027 defense target depends on evidence and committee approval. CS guidance calls for an advisor-approved complete dissertation to reach committee members two weeks before the defense [29]. The Graduate School lists February 5, 2027 for the graduation application and doctoral candidacy form, and March 26, 2027 for the defended final ProQuest submission [30]; these dates must be rechecked before filing. If core end-to-end evidence is missing at the January review, scope or timing will be reconsidered with the committee.

6.2 Limits That Must Remain Visible

ABE adds authority, computation, and rekeying costs; disconnected revocation has propagation limits. Candidate ACKs do not prove future capacity or honest reporting. Cache availability does not guarantee task completion. Tests confined to one process do not establish distributed failure behavior. UAV safety and perception accuracy need evidence beyond service-message delivery. Findings from selected hardware and models must be interpreted within those configurations.

Chapter 7

Expected Outcomes and Proposal Summary

The research builds on NDNF's initial ability to discover multiple candidates and select one Provider for an invocation. It evaluates that discovery/authorization foundation and investigates an extension for complementary Provider roles and named dependencies. The intended contribution is an evaluated service protocol and design rationale, not a new encryption primitive or the first use of named computation. RQ1 compares ABE-backed authorization with DNMP-inspired role rules paired with content encryption, including their management and withdrawal costs. RQ2 asks how runtime assignments become local rules for accepting named dependencies. RQ3 examines benefits and costs for invocation and collaboration separately. Existing campaigns establish limited invocation findings; the collaboration and cross-application claims still require their declared end-to-end evidence. The proposal therefore asks the committee to assess the research scope, comparison designs, validation criteria, and feasibility of the completion plan. The dissertation will report supported answers and limitations, including conditions in which the proposed mechanisms offer no advantage.

References

- [1] L. Zhang, A. Afanasyev, J. Burke, V. Jacobson, kc claffy, P. Crowley, C. Papadopoulos, L. Wang, B. Zhang, Named data networking, in: ACM SIGCOMM Computer Communication Review, 2014, pp. 66–73.
- [2] Named Data Networking Project, NDN Packet Format Specification: Data Packet, <https://docs.named-data.net/NDN-packet-spec/current/data.html>, , accessed September 9, 2026.
- [3] Y. Yu, A. Afanasyev, D. Clark, k. claffy, V. Jacobson, L. Zhang, Schematizing trust in named data networking, in: Proceedings of the 2nd ACM Conference on Information-Centric Networking, 2015, pp. 177–186. doi:10.1145/2810156.2810170.
- [4] T. Li, W. Shang, A. Afanasyev, L. Wang, L. Zhang, A brief introduction to ndn dataset synchronization (ndn sync), in: 2018 IEEE Military Communications Conference, 2018, pp. 612–618.

- [5] P. Moll, V. Patil, L. Wang, L. Zhang, Sok: The evolution of distributed dataset synchronization solutions in ndn, in: Proceedings of the 9th ACM Conference on Information-Centric Networking, 2022, pp. 33–44.
- [6] C. Tschudin, M. Sifalakis, Named functions and cached computations, in: 2014 IEEE 11th Consumer Communications and Networking Conference, 2014, pp. 851–857.
- [7] M. Krol, I. Psaras, Nfaas: Named function as a service, in: Proceedings of the 4th ACM Conference on Information-Centric Networking, 2017, pp. 134–144.
- [8] M. Krol, K. Habak, D. Oran, D. Kutscher, I. Psaras, Rice: Remote method invocation in icn, in: Proceedings of the 5th ACM Conference on Information-Centric Networking, 2018, pp. 1–11.
- [9] T. Meirovitch, L. Zhang, Nsc–named service calls, or a remote procedure call for ndn, in: NDN Technical Report NDN-0074, Revision 1, 2021, pp. 1–20.
- [10] K. Nichols, Lessons learned building a secure network measurement framework using basic ndn, in: Proceedings of the 6th ACM Conference on Information-Centric Networking, 2019, pp. 112–122. doi:10.1145/3357150.3357397.
- [11] grpc: A high performance, open source universal rpc framework, <https://grpc.io/>, , accessed 2026.
- [12] M. Quigley, K. Conley, B. Gerkey, J. Faust, T. Foote, J. Leibs, R. Wheeler, A. Y. Ng, Ros: an open-source robot operating system, ICRA Workshop on Open Source Software.
- [13] Ros 2 documentation, <https://docs.ros.org/>, , accessed 2026.
- [14] V. John, X. Liu, A survey of distributed message broker queues, arXiv preprint arXiv:1704.00411.
- [15] M. S. ud din, M. A. ur Rehman, B.-S. Kim, Cfec: An ultra-low latency microservices-based in-network computing framework for information-centric iovs, IEEE Transactions on Services Computing 17 (6) (2024) 3199–3212.
- [16] M. Imran, et al., Mia-ndn: Microservice-centric interest aggregation in named data networking, Sensors 23 (3) (2023) 1411.
- [17] R. Tourani, et al., Towards security-as-a-service in multi-access edge, in: Proceedings of the 4th ACM/IEEE Symposium on Edge Computing, 2019, pp. 358–363.
- [18] C. Barrios, E. Alker, M. Kumar, CaSCON: Cache-assisted service composition orchestration leveraging NDN, in: 2025 IEEE International Conference on Smart Computing (SMARTCOMP), 2025, pp. 122–129. doi:10.1109/SMARTCOMP65954.2025.00061.

- [19] S. Dulal, T. Yu, S. Liu, A. R. Thieme, L. Zhang, L. Wang, Enhancing NAC-ABE to support access control for mHealth applications and beyond, arXiv:2311.07299 (2023). doi:10.48550/arXiv.2311.07299.
URL <https://arxiv.org/abs/2311.07299>
- [20] A. Koubâa, A. Allouch, M. Alajlan, Y. Javed, A. Belghith, M. Khalgui, Micro air vehicle link (mavlink) in a nutshell: A survey, IEEE Access 7 (2019) 87658–87680. doi:10.1109/ACCESS.2019.2924410.
- [21] Px4 autopilot user guide, <https://docs.px4.io/>, , accessed 2026.
- [22] Minindn: A mininet-based ndn emulator, <https://github.com/named-data/mini-ndn>, , accessed 2026.
- [23] ONNX Runtime Authors, How to configure the past present share buffer, <https://onnxruntime.ai/docs/genai/howto/past-present-share-buffer.html>, , accessed September 11, 2026.
- [24] Open neural network exchange (onnx), <https://github.com/onnx/onnx>, , accessed 2026.
- [25] Y. Kang, J. Hauswald, C. Gao, A. Rovinski, T. Mudge, J. Mars, L. Tang, Neurosurgeon: Collaborative intelligence between the cloud and mobile edge, in: Proceedings of the Twenty-Second International Conference on Architectural Support for Programming Languages and Operating Systems, 2017, pp. 615–629. doi:10.1145/3037697.3037698.
- [26] S. Teerapittayanon, B. McDanel, H. T. Kung, Distributed deep neural networks over the cloud, the edge and end devices, in: 2017 IEEE 37th International Conference on Distributed Computing Systems, 2017, pp. 328–339. doi:10.1109/ICDCS.2017.226.
- [27] NDNsf Project, Proposal research revision: Claim and evidence audit, Companion repository document: docs/PAPER/proposal-defense/research-revision-audit.md, , september 9, 2026; includes frozen campaign paths, evidence limits, and unresolved research gates.
- [28] gRPC Authors, Custom name resolution, <https://grpc.io/docs/guides/custom-name-resolution/>, , accessed September 9, 2026.
- [29] University of Memphis, Department of Computer Science, Information for computer science ph.d. students, https://www.memphis.edu/cs/pdfs/forms_grad_phd_programinfo.pdf, , accessed September 9, 2026.
- [30] University of Memphis Graduate School, Graduate school calendar, <https://www.memphis.edu/gradschool/news-events/calendar.php>, , spring 2027 calendar, checked September 14, 2026.